Lab 9: Dynamic Programming

Data Structures & Algorithms

Time Limit: 2 seconds per problem — Language: C++

Instructions: Solve all three problems below using Dynamic Programming. Each problem **must** be implemented as a C++ class with the interface specified. All three problems are handled in a **single program**: the first line of the input is a problem indicator (`1`, `2`, or `3`) that tells `main()` which problem to solve. After the problem data, a series of **commands** follow, one per line, until the keyword `end` is encountered. **Solutions that exceed the time limit will receive zero marks, regardless of correctness.**

Command Protocol (applies to all three problems):

- `solve` — run the DP and print **only the final answer** (a single number on its own line).
- `backtrace` — print the selection (selected items, robbed houses, etc.). Valid only *after* `solve` has been issued.
- `end` — stop reading commands and terminate the program.

Input Routing and Command Loop:

```
int main() {
    int problem;
    cin >> problem;
    if (problem == 1)
    {
        /* read Knapsack input, create object, execute commands
           */
        return 0;
    }
    if (problem == 2)
    {
        /* read HouseRobber input, create object, execute
           commands */
        return 0;
    }
    if (problem == 3)
    {
        /* read TwoSets input, create object, execute commands */
        return 0;
    }
}

 ********************************************************
 *  You can use the below snippet for parsing command   *
 ********************************************************
    string cmd;
    while (cin >> cmd && cmd != "end") {
        if (cmd == "solve")     { /* call solve() and prints
```

```
            final answer only */ }
        if (cmd == "backtrace") { /* call backtrace() and prints
            recovered solution*/ }
    }
```

Problem 1: 0/1 Knapsack

Problem Statement

You are given N items, each with a **weight** and a **value**, and a knapsack with a maximum weight capacity W. Select items such that the total weight does not exceed W and the **total value is maximized**. Each item can either be picked entirely or left behind — partial selection is **not** allowed.

Required Class Interface

You must implement the following class as specified. Do **not** change method names, return types, or parameter types. You can add your own helper functions and variables in the class.

```
struct Item {
    int weight;
    int value;
};

class Knapsack {
private:
    int capacity;
    vector<Item> items;
    vector<vector<int>> dp;  // DP table

public:
    // Constructor: sets knapsack capacity
    Knapsack(int capacity);

    // Adds an item to the item list
    void addItem(int weight, int value);

    // Builds the DP table and returns the maximum value (single
       integer)
    int solve();

    // Prints the full DP table row by row (for debugging)
    void print();

    // Backtracks through the DP table; it first number of items
       selected and then on the same line prints selected item
    // indices (0-indexed, space-separated, ascending order)
    void backtrace();
};
```

Class Requirements:

- `Knapsack(int capacity)` — initializes the knapsack with the given weight capacity.
- `addItem(int weight, int value)` — appends a new item to the internal item list.
- `solve()` — builds the complete $dp[i][w]$ table and returns the maximum value.
- `print()` — prints the full DP table row by row (space-separated values per row). Must be called only *after* `solve()`.
- `backtrace()` — backtracks through the DP table to recover chosen item indices; it first prints number of chosen items and then on the same line prints the indices of the chosen item as space-separated 0-indexed integer in ascending order. Must be called only *after* `solve()`.

DP Definition

Define $dp[i][w]$ as the maximum value achievable using the first i items with knapsack capacity w.
While reconstructing the solution from the DP table(for backtrace command), if both including the i-th item and excluding it yield the same value, always prefer excluding the i-th item. This ensures a deterministic output.

Input Format

- Line 1: `1` (problem indicator).
- Line 2: two integers N and W.
- Next N lines: two integers `weight` and `value` per item.
- Remaining lines: commands (`solve`, `backtrace`) terminated by `end`.

Output Format

- `solve` → print the maximum value as a single integer.
- `backtrace` → print the number of selected items and their indices space-separated in ascending order.

Example

Input

```
1
4 5
1 1
2 6
3 10
5 16
solve
backtrace
end
```

Output

```
16
2 1 2
```

The DP table (rows = items 0…N, cols = capacity 0…W):

Item \ w	**0**	**1**	**2**	**3**	**4**	**5**
0 (w=1, v=1)	0	1	1	1	1	1
1 (w=2, v=6)	0	1	6	7	7	7
2 (w=3, v=10)	0	1	6	10	11	16
3 (w=5, v=16)	0	1	6	10	11	**16**

Sample Test Cases

N	**W**	**Weights**	**Values**	**Answer (solve)**
4	5	[1, 2, 3, 5]	[1, 6, 10, 16]	16
3	4	[1, 3, 4]	[1, 4, 5]	5
3	6	[2, 3, 4]	[3, 4, 5]	8
1	10	[15]	[20]	0

Constraints:

- $1 \leq N \leq 1000, \quad 1 \leq W \leq 1000$
- $1 \leq \text{weight}[i] \leq 1000, \quad 1 \leq \text{value}[i] \leq 10^4$
- **Time Limit: 2 seconds. Exceeding the time limit results in zero marks.**

Problem 2: House Robber

Problem Statement

You are a robber planning to rob houses along a street. Each house has a certain amount of money. Adjacent houses are connected to a shared security system — **you cannot rob two directly adjacent houses** on the same night without triggering the alarm.

Given an array of non-negative integers representing the money in each house, determine the **maximum amount you can rob** without triggering the alarm. Robbing a house with zero money is invalid, your sequence of robbed house should not include house with zero money.

Required Class Interface

You must implement the following class as specified. Do **not** change method names, return types, or parameter types. You can add your own helper functions and variables in the class.

```
class HouseRobber {
private:
    vector<int> houses;
    vector<int> dp;  // DP array

public:
    // Constructor: initializes with the list of house values
    HouseRobber(vector<int> houseValues);

    // Builds the DP array and returns the maximum amount
    int solve();

    // Prints the full DP array space-separated (for debugging)
    void print();

    // Backtracks through the DP array; it first prints the
       number of robbed houses and then on the same line prints
       robbed houses indices (0-indexed, space-separated,
       ascending order)
    void backtrace();
};
```

Class Requirements:

- `HouseRobber(vector<int> houseValues)` — stores the house money array internally.
- `solve()` — fills the `dp` array bottom-up and returns the maximum amount.
- `print()` — prints the full `dp` array space-separated on a single line. Must be called only *after* `solve()`.
- `backtrace()` — backtracks through the `dp` array to recover robbed house indices; it first prints number of robbed houses and then on the same line prints house indices of robbed houses as space-separated 0-indexed integer in ascend-

ing order. Must be called only *after* `solve()`.

DP Definition

Define $dp[i]$ as the maximum money robbed from the first i houses.
While reconstructing the solution from the DP table(for backtrace command), if both robbing the i-th house and not robbing it yield the same value, always prefer not robbing the i-th house. This ensures a deterministic output.

Input Format

- Line 1: `2` (problem indicator).
- Line 2: integer N — the number of houses.
- Line 3: N space-separated non-negative integers.
- Remaining lines: commands (`solve`, `backtrace`) terminated by `end`.

Output Format

- `solve` → print the maximum amount as a single integer.
- `backtrace` → print robbed house indices space-separated in ascending order.

Example

Input

```
2
5
2 7 9 3 1
solve
backtrace
end
```

Output

```
12
3 0 2 4
```

Input

```
2
6
5 5 10 100 10 5
solve
backtrace
end
```

Output

```
110
3 0 3 5
```

The DP array trace for this example:

i	0	1	2	3	4
houses$[i]$	2	7	9	3	1
dp$[i]$	2	7	11	11	**12**

Sample Test Cases

N	Houses	Answer (solve)
5	[2, 7, 9, 3, 1]	12
4	[1, 2, 3, 1]	4
6	[5, 5, 10, 100, 10, 5]	110
1	[10]	10
2	[1, 2]	2
3	[0, 0, 0]	0

Constraints:

- $1 \leq N \leq 10^5, \quad 0 \leq \text{houses}[i] \leq 10^4$
- **Time Limit: 2 seconds. Exceeding the time limit results in zero marks.**

Problem 3: Two Sets

Problem Statement

You are given the numbers $1, 2, \ldots, n$. Your task is to divide these numbers into **two sets** of equal sum. Count the number of ways this can be done, modulo $10^9 + 7$.

Two divisions are considered different only if the sets contain different elements.

Note: If the total sum $S = \frac{n(n+1)}{2}$ is odd, it cannot be split equally, so the answer is `0`.

Required Class Interface

You must implement the following class as specified. Do **not** change method names, return types, or parameter types. You can add your own helper functions and variables in the class.

```
const int MOD = 1e9 + 7;

class TwoSets {
private:
    int n;
    // your variables for dp implementation

public:
    // Constructor: stores n, initializes data members
    TwoSets(int n);

    // Builds the DP table and returns the answer
    // Returns 0 immediately if a valid split is not possible
    int solve();

    // Prints the full DP table row by row (for debugging)
    // Must be called only after solve()
    void print();

    // No backtrace() required for this problem
};
```

Class Requirements:

- `TwoSets(int n)` — stores n and initializes all data members.
- `solve()` — returns `0` immediately if a valid split is not possible. Otherwise fills the DP table and returns the answer as a single integer.
- `print()` — prints the full DP table row by row (space-separated values per row). Must be called only *after* `solve()`.
- No `backtrace()` is required for this problem. The `end` keyword still terminates input.

DP Definition

Try to come up with your own DP definition to solve this problem :)

Input Format

- Line 1: `3` (problem indicator).
- Line 2: a single integer n.
- Remaining lines: commands (`solve`, `print`) terminated by `end`. (`backtrace` is not applicable.)

Output Format

- `solve` → print the answer as a single integer.
- `print` → print the full DP table row by row (space-separated).

Examples

Input 1

```
3
7
solve
end
```

Output 1

```
4
```

Input 2

```
3
6
solve
end
```

Output 2

```
0
```

Sample Test Cases

n	Answer (solve)
2	0
3	1
4	1
7	4

For example, when $n = 7$, there are 4 valid partitions:

$$\begin{aligned} \{1,3,4,6\} \quad &\text{and} \quad \{2,5,7\} \\ \{1,2,5,6\} \quad &\text{and} \quad \{3,4,7\} \\ \{1,2,4,7\} \quad &\text{and} \quad \{3,5,6\} \\ \{1,6,7\} \quad &\text{and} \quad \{2,3,4,5\} \end{aligned}$$

when $n = 3$, there is 1 valid partition:

$$\{1,2\} \quad \text{and} \quad \{3\}$$

Constraints:

- $1 \leq n \leq 500$
- Output modulo $10^9 + 7$
- **Time Limit: 2 seconds. Exceeding the time limit results in zero marks.**

Submission Checklist:

☐ All three classes implemented with the **exact** method signatures given above.
☐ `main()` reads the problem indicator and routes to the correct class.
☐ Command loop correctly handles `solve`, `backtrace`, and `end`.
☐ `solve` prints **only** the final answer (a single integer); no extra labels or text.
☐ `print` prints the full DP table / array; `backtrace` prints only the recovered solution.
☐ Code compiles without warnings using `g++ -std=c++17`.
☐ Solution runs within the 2-second time limit — **TLE results in zero marks**.

Testcases Distribution - Total 50 tcs

- Problem 1 - 0/1 Knapsack (10 solve + 10 solve & backtrace testcases)
- Problem 2 - House Robber (10 solve + 10 solve & backtrace testcases)
- Problem 3 - Two Sets (10 solve testcases)